

Internet & Layers

- **Hosts/edge**: end systems run apps. **Core**: routers, links. **Access**: DSL, cable, FTTH, cellular, WiFi.
- **Packet switching**: store-and-forward, statistical mux, no setup, may queue/drop. **Circuit switching**: reserved BW (FDM/TDM), guaranteed rate, idle waste.
- **Why packet wins**: bursty traffic → high utilisation. Ex: 1Gbps link, 100Mbps users at 10% duty → packet supports many more users than CS.
- **5-layer stack** (TCP/IP): Application · Transport · Network · Link · Physical. OSI adds Session, Presentation.
- **Encapsulation**: each layer prepends header. msg→seg→datagram→frame→bits.
- **Standards bodies**: IETF (RFCs/TCP-IP), ISO (OSI), IEEE 802 (LAN/WiFi), W3C (web).

Delay & Throughput

Nodal delay = d_{proc} + d_{queue} + d_{trans} + d_{prop}

- d_{trans} = L/R (packet L bits, link rate R bps)
- d_{prop} = d/s (link length / signal speed ≈ 2·10⁸ m/s)
- Queueing: traffic intensity a = La/R; →1 ⇒ delay blows up.
- **End-to-end** through N store-and-forward links of rate R: t = (N·L)/R (+propagation).
- **Throughput**: min(R_s, R_c, R/N) bottleneck link.
- **Bandwidth-delay product**: max bits "in flight" = R·RTT.

Ex: 350MB over 2Mbps = 8·350·10⁶/(2·10⁶) = 1400 s. Throughput ≈ 2Mbps. Prop (100ms RTT) negligible vs transfer.

P2P Distribution Time

File F to N peers. Server upload u_s, peer up u_i, down d_i.

- **Client–server**: D_{cs} = max(N·F/u_s, F/min(d_i))
- **P2P**: D_{p2p} = max(F/u_s, F/min(d_i), N·F / (u_s+Σu_i))
- P2P scales with N; CS grows linearly with N.
- **BitTorrent**: torrent file, tracker, chunks (256KB), rarest-first, tit-for-tat unchoke (top-4 + optimistic).

HTTP

- TCP port **80** (HTTPS 443). Stateless. Request: method URL HTTP/1.1, headers, body.
- **Methods**: GET, POST, HEAD, PUT, DELETE, OPTIONS.
- **Status**: 1xx info · 2xx OK (200) · 3xx redirect (301, 304 Not Modified) · 4xx client (400, 404) · 5xx server (500, 503).
- **Non-persistent**: 1 obj/TCP = 2 RTT + transmit. **Persistent** (HTTP/1.1, default): reuse TCP. **Pipelining**: multiple req without waiting.
- **Cookies**: Set-Cookie ↔ Cookie header; identifies stateful sessions over stateless HTTP.
- **Conditional GET**: If-Modified-Since → 304 if cached copy fresh.
- **Caches/proxies**: cut response time & WAN traffic. CDN: place content near edge.
- HTTP/2 binary, multiplexed streams. HTTP/3 over QUIC/UDP.

DNS

- Port **53**, mostly UDP. Hierarchical: root → TLD (.uk, .com) → authoritative.
- **Queries**: recursive (local resolver does work) vs iterative (referral chain).
- **Records**: A (IPv4), AAAA (IPv6), NS (name server), CNAME (alias), MX (mail), PTR (reverse), TXT.
- Caching with TTL at local DNS; reduces load on root/TLD.
- DNS used for CDN redirection (return nearest server's IP).

Email & Other App

- **SMTP** 25 (push, ASCII, persistent). **POP3** 110, **IMAP** 143 (server-side state).

- **FTP**: control 21 (persistent), data 20 (new conn/file). Out-of-band control.
- **Streaming: DASH** — server splits video into chunks at multiple rates; client picks rate per chunk based on measured bandwidth.

Sockets (Python)

```

TCP server
s=socket(AF_INET,SOCK_STREAM)
s.bind((host,port)); s.listen(1)
c,addr=s.accept(); c.recv(1024); c.send(d)

TCP client
s=socket(AF_INET,SOCK_STREAM)
s.connect((h,p)); s.send(d); s.recv(1024)

UDP
s=socket(AF_INET,SOCK_DGRAM)
s.sendto(d,(h,p));
d,addr=s.recvfrom(1024)
    
```

TCP: one socket per client (demux by 4-tuple). UDP: single socket (demux by dst port only).

Transport: UDP vs TCP

UDP	TCP
connectionless	3-way handshake
no RDT	reliable, in-order
no flow/cong ctrl	flow + congestion
8-byte hdr	20-byte (no opts)
DNS, RTP, QUIC, games	HTTP, SMTP, FTP, SSH

UDP hdr: src port (16), dst port (16), length (16), checksum (16).

Why UDP? low latency, no setup, no cwnd throttling, multicast, header tiny, app-level control.

Mux / Demux

- **UDP**: demux by 2-tuple (dst IP, dst port).
- **TCP**: demux by 4-tuple (src IP, src port, dst IP, dst port). Server creates new socket per accepted conn.

Internet Checksum

- Treat data as 16-bit words.
 - Sum with end-around carry (1's complement add).
 - Take 1's complement → checksum.
 - Receiver: sum all incl. checksum = 0xFFFF if OK.
- Ex 0xf351+0x8040+0x2010+0x1b09 = 0x1AEAA → fold → 0xAEAB; ~ = 0x5154.

Reliable Data Transfer

- **rdt1.0** perfect channel.
- **rdt2.0** add ACK/NAK + checksum.
- **rdt2.1/2.2** add seq# (1 bit) to detect dup ACKs.
- **rdt3.0** add timer (loss). Stop-and-wait — utilisation U = (L/R)/(RTT+L/R) → tiny.
- **Pipelining**: increase utilisation.

	Go-Back-N	Sel. Repeat
ACK	cumulative	individual
retransmit	from N	only lost
buffer rx	no	yes
win ≤	2 ^k −1	2 ^k −1

TCP

Header (20B base): src/dst port (16+16), seq# (32, byte-based), ack# (32), data offset (4), flags (URG/ACK/PSH/RST/SYN/FIN), **rwnd** (16), checksum (16), urg ptr (16), options.

3-way handshake: SYN seq=x → SYN-ACK seq=y ack=x+1 → ACK ack=y+1.

Teardown: FIN → ACK → FIN → ACK (each side closes independently).

Cumulative ACK: ACK n ⇒ all bytes < n received. 3 dup-ACKs ⇒ fast retransmit.

RTT estimation:

- EstRTT = (1−α)·EstRTT + α·SampleRTT; α=0.125
- DevRTT = (1−β)·DevRTT + β·|Sample−Est|; β=0.25
- Timeout = EstRTT + 4·DevRTT (doubled on retransmit)

Flow control: sender keeps unacked ≤ rwnd advertised by receiver.

TCP Congestion Control

- Effective wnd = min(cwnd, rwnd). LastByteSent – LastByteAcked ≤ wnd.
- **Slow start**: cwnd doubles per RTT (cwnd += MSS per ACK) until ssthresh.
- **Cong. avoidance**: cwnd += MSS·MSS/cwnd per ACK (linear, +1 MSS/RTT). AIMD.
- **Loss = 3 dup ACKs (TCP Reno)**: ssthresh=cwnd/2; cwnd=ssthresh; fast recovery.
- **Loss = timeout (Tahoe)**: ssthresh=cwnd/2; cwnd=1 MSS; slow start.
- **Throughput** ≈ 1.22·MSS/(RTT·√L) for loss rate L.
- **Fairness**: AIMD converges to equal share among TCP flows sharing bottleneck.
- **QUIC**: UDP-based, 0-RTT, per-stream loss (no HoL block at transport), encryption built-in.

Worked Throughput Example

350 MB film, 2 Mbps link, RTT 100 ms:

- Bits = 350·10⁶·8 = 2.8·10⁹.
- Tx time = 2.8·10⁹/2·10⁶ = 1400 s ≈ 23 min.
- Throughput ≈ 2 Mbps (bottleneck). RTT adds 100 ms — negligible vs 1400 s.

P2P tabulation (F=10Gb, u_s=50Mbps, d=2Mbps):

N, u	CS	P2P
10, 0.25M	2000s	≈5000s
10, 0.5M	2000s	≈3636s
100, 0.25M	20000s	≈13333s
100, 0.5M	20000s	≈10000s

TCP Connection Lifecycle

State machine (client): CLOSED → SYN_SENT → ESTABLISHED → FIN_WAIT_1 → FIN_WAIT_2 → TIME_WAIT (2·MSL) → CLOSED.

Server: LISTEN → SYN_RCVD → ESTABLISHED → CLOSE_WAIT → LAST_ACK → CLOSED.

- TIME_WAIT prevents stale segs from confusing a new connection on the same 4-tuple.
- Half-close possible: each direction FINs separately.

TCP Sliding Window

- Sender keeps: SendBase (oldest unacked), NextSeqNum.
- Effective window = min(cwnd, rwnd). Sender may have ≤ window bytes outstanding.
- Cumulative ACK: ACK=n means "all bytes < n received, expecting n next".
- Receiver may buffer out-of-order data and ACK highest in-order byte.
- **Nagle**: coalesce small writes when prior data unacked.
- **Delayed ACK**: receiver waits ≤500 ms to piggyback ACK on data (one ACK per 2 segs typical).

CDN & Server Selection

- **Enter deep**: many servers, close to users (Akamai).
- **Bring home**: few large centres (Limelight).
- DNS-based redirection: authoritative DNS for CDN returns IP of "best" replica based on requester's local DNS IP (proxy for location), load, RTT.
- Benefits: lower latency, lower origin load, resilience.

TLS / HTTPS Basics

- Sits between application and TCP. Port 443.
- **Handshake**: ClientHello (versions, ciphers, random) → ServerHello + cert → key exchange (RSA / DHE / ECDHE) → Finished. TLS 1.3 = 1-RTT (0-RTT resumption).
- Provides: confidentiality (AES-GCM), integrity (HMAC), authentication (X.509 cert chain to CA).
- HTTPS = HTTP over TLS over TCP.

IPv4 Datagram

Header 20–60B. Fields (bits): Ver (4)=4, IHL (4), TOS/ DSCP (8), Total Length (16), ID (16), Flags (3: DF/MF), Frag Offset (13, in 8-byte units), TTL (8), Protocol (8: 6=TCP, 17=UDP, 1=ICMP), Checksum (16), Src IP (32), Dst IP (32), Options.

Fragmentation: by router when MTU smaller; reassembly at destination. IPv6 only sources fragment.

Addressing & CIDR

- Dotted decimal /n e.g. 192.168.1.0/24 → 2^{32−n} addrs.
- Subnet** = group of interfaces sharing prefix; reach each other w/o router.
- Longest prefix match** at forwarding table.
- Private:** 10/8, 172.16/12, 192.168/16. Loopback 127/8.
- NAT:** rewrites (src IP, src port) → (public IP, new port); table maps. Cons: breaks end-to-end, hole-punching needed.
- DHCP** (UDP 67/68) — **DORA:** Discover (bcast) → Offer → Request → ACK; gives IP, mask, GW, DNS, lease.
- IPv6:** 128-bit, fixed 40B header, no checksum, no in-router frag, extension headers; transition via dual-stack & tunneling (6-in-4).

Router & SDN

- Components: input ports, switch fabric, output ports, routing processor. **HoL blocking** at input queues.
- Forwarding** (data plane, ns timescale) vs **Routing** (control plane, sec).
- SDN:** logically central controller pushes flow tables. **OpenFlow:** match (header fields) + action (forward/ drop/modify/encap).

Routing Algorithms

	Link State	Distance Vector
info	full topology	only neighbours
algo	Dijkstra	Bellman–Ford
msg	flood LSAs	exchange DV
complexity	O(n²) / O(n log n)	iterative
issues	OSPF DR/BDR	count-to-∞

Count-to-∞ fix: split horizon (don't advertise back), poisoned reverse (advertise ∞).

Dijkstra step: pick unvisited node with min D; relax neighbours D(v)=min(D(v), D(u)+c(u,v)).

Intra/Inter-AS

- Intra-AS** (IGP): **OSPF** (link-state, areas — backbone + area routers, supports hierarchy/auth/multipath), **RIP** (DV, hop count, max 15, slow).
- Inter-AS:** **BGP** over TCP/179. **eBGP** between ASes, **iBGP** inside AS. Attributes: **AS-PATH** (loop detect), **NEXT-HOP**, **LOCAL-PREF**, **MED**.
- Hot-potato:** pick eBGP exit with smallest intra-AS cost.
- Policy-driven; not shortest-path.

Hierarchical Routing Sizing

n areas, R routers each. Table size at each router ≈ R + (n−1) routes for foreign areas.

Ex: 1000 routers, 20 areas → 50+19 = 69 entries. Solve 50·k + 19 ≤ 100 → table cap 100 ⇒ max routers/area ≈ (100−n+1). With 20 areas, R≤81 ⇒ total ≤ 1620.

ICMP

- Network-layer signalling over IP (proto 1).
- Types: 0 echo reply, 8 echo req (**ping**); 3 dst unreachable; 11 **TTL exceeded** (used by **traceroute**: send UDP with increasing TTL).

Link Layer Services

- Framing, link access (MAC), optional reliable delivery & flow ctrl, error detect/correct.
- Parity:** 1-bit detects single error; **2D parity** detects+corrects single bit.
- CRC:** D·2^l mod G; receiver checks remainder. Detects all burst ≤ r.
- Internet checksum:** 1's complement; lighter than CRC.

MAC Protocols

- Channel partition:** TDMA, FDMA, CDMA — efficient under load, wasteful idle.

• Random access:

- ALOHA: efficiency max 1/(2e) ≈ 18%; slotted ALOHA 1/e ≈ 37%.
- CSMA: listen before transmit; CSMA/CD: also abort on collision.
- Binary exponential backoff:** after n-th collision, wait k slots, k∈{0..2^{min(n,10)}−1}.
- CSMA/CA (WiFi): can't detect collision on radio; uses ACK, RTS/CTS optional.

- Taking turns:** polling, token passing.

Ethernet

- Frame: **Preamble** 8B (7×0x55 + SFD 0xD5), **Dst** 6B, **Src** 6B, **Type/Length** 2B (0x0800=IPv4, 0x0806=ARP), **Payload** 46–1500B, **CRC** 4B. MTU 1500.
- MAC addr 48 bits, globally unique (OUI+NIC). Broadcast FF:FF:FF:FF:FF:FF.
- Half-duplex shared (legacy) uses CSMA/CD; modern switched full-duplex doesn't collide.
- Connectionless, unreliable (no ACK); CRC drops corrupt frames.

Switches & ARP

- Switch:** layer-2, transparent, **self-learning** — records (src MAC, port, TTL) on each frame; forwards by lookup, floods unknown. Plug-and-play, full-duplex per port, no collisions.
- Host roams: new frame from new port refreshes table → traffic follows.
- ARP** (within subnet): request broadcast "who has IP X?" → owner replies unicast with MAC. Cached with TTL.
- Off-subnet: send to default gateway's MAC; gateway re-ARPs next hop.
- VLANS:** partition switch into virtual LANs; 802.1Q tag 4B (VID 12 bits).
- Switches vs Routers:** switch=L2 plug-and-play, no path optimisation; router=L3, isolates broadcast, runs routing.

WiFi 802.11 (brief)

- CSMA/CA + DIFS/SIFS + ACK; RTS/CTS for hidden terminal.
- Frame has 4 addr fields (BSS routing). AP = bridge to wired LAN.

Congestion: Effects & Approaches

- Effects:** queue delay ↑, loss/drops, retransmissions waste BW, throughput collapse, fairness loss.
- End-end** (TCP): infer loss → AIMD.
- Network-assisted:** ECN bits in IP (RFC 3168), routers mark instead of drop.
- Approaches: avoidance (admission, traffic shaping/ leaky-bucket), reaction (AIMD).

Exam Heuristics

- Throughput Qs:** bottleneck = min(R_i); transfer time = F/bottleneck (+RTT for setup).
- Optimal packet size** p with header h, k hops, x bits: total ≈ (x/p)·(p+h)/b + (k−1)(p+h)/b. Minimise → p = √((k−1)·h·x · something) ... in practice differentiate (x·(p+h)/(p·b)) + k·(p+h)/b w.r.t. p, set =0 ⇒ p = √(h·x/(k−1)) approximately.
- 2 disjoint paths:** run Dijkstra, remove edges of path1, run again — Suurballe-like heuristic.
- Subnets ↔ hierarchical routing:** subnet prefix aggregates many hosts into one routing entry → same scaling trick at any level.
- Standards pros:** interop, competition, lower cost.
- Cons:** slow to evolve, lowest-common-denominator, lock-in.
- Compression tradeoff:** comp+tx time = T_c + F(1−r)/R. Equal to uncompressed F/R when R = F·r / T_c. Propagation delay unchanged (depends on distance, not data).

Quick Numbers

Ports	HTTP 80 · HTTPS 443 · DNS 53 · SMTP 25 · IMAP 143 · POP3 110 · FTP 21/20 · SSH 22 · DHCP 67/68 · BGP 179
Sizes	UDP hdr 8B · TCP hdr 20B · IPv4 hdr 20B · IPv6 hdr 40B · MAC 48b · IPv4 32b · IPv6 128b · Eth payload 46–1500B
Speeds	1 Mbps=10 ⁶ bps · prop ≈ 2·10 ⁸ m/s in fibre · 1 byte=8 bits
Efficiency	ALOHA 1/(2e) · S-ALOHA 1/e · S&W ≪1 · TCP fair AIMD

NAT Details & Traversal

- NAT box keeps a translation table: (LAN IP, LAN port) ↔ (WAN IP, WAN port).
- Outgoing pkt: rewrite src to (WAN IP, new port), add entry. Incoming reply: reverse lookup.
- Critiques: violates end-to-end principle, breaks P2P/ incoming, port limits (2¹⁶ per WAN IP).
- Traversal:** relaying (TURN), reverse connection, hole-punching via rendez-vous server (STUN/ICE).

Switching: Spanning Tree

- Loops in switched LAN ⇒ broadcast storms. **STP 802.1D** builds loop-free tree.
- Elect root bridge (lowest BID). Each switch picks root port (lowest path cost). Other ports designated/ blocked.
- BPDUs carry root ID + cost.
- RSTP 802.1w** = fast reconvergence.

MPLS & Data Centres

- MPLS:** 32-bit shim header between L2/L3. Label-switched paths; routers (LSRs) forward on 20-bit label. Enables traffic engineering, VPN.
- Data centre:** hierarchical fat-tree topology (TOR → aggregate → core); ECMP load-balance; oversubscription at higher tiers.
- Containerised servers, virtual switches; east-west traffic dominates.

Multimedia & Streaming

- UDP+RTP for real-time (VoIP, video conf). RTP carries timestamp + seq#.
- Jitter:** variation in pkt delay. Mitigated by playout buffer.
- FEC:** send redundant info to recover loss without retransmit.
- DASH:** HTTP-based adaptive streaming; client measures TCP throughput, fetches next chunk at best rate.
- CDN:** server selection via DNS — geographic, load, network distance.

Security Quick Facts

- Symmetric:** AES (fast, shared key). **Asymmetric:** RSA, ECC (key exchange, sigs).
- Hash:** SHA-256 (integrity). MAC = hash with key (HMAC).
- Digital sig:** sign hash with private key; verify with public key.
- Cert:** CA signs (subject pubkey, identity). X.509.
- Attacks: sniffing, IP spoofing, DDoS (volumetric/ protocol), DNS poisoning, BGP hijacking, MITM (blocked by TLS+cert pinning).
- Firewall:** filters by (IP, port, proto). Stateful firewall tracks connections. IDS = passive detection.

Reasoning Patterns

- Why packet over circuit?** Bursty users: probability that K of N are active simultaneously is binomial; for N≫K with low duty, prob >K active is tiny ⇒ statistical mux works.
- Optimal packet size:** total delay D(p) = (k−1)(p+h)/b (pipeline fill) + (x/p)(p+h)/b. dD/dp=0 ⇒ p* = √((k−1)·h·x) form depends on derivation; key insight: small p ⇒ header overhead high, big p ⇒ pipeline fills slow.
- Two disjoint paths:** 1) Dijkstra → P1. 2) Remove P1 edges. 3) Dijkstra → P2. Survives any single link loss.
- Self-learning vs roaming:** when host moves to new port, first frame from new port refreshes table entry (port column updated), so reverse traffic now follows.
- Per-pkt ACK vs per-file ACK:** per-pkt = recovery is fast (only retransmit one) but high overhead; per-file = low overhead but single loss ⇒ retransmit entire file.